Inteligență Artificială

Probleme de căutare

Slides: Andrei Olaru, Adina Florea

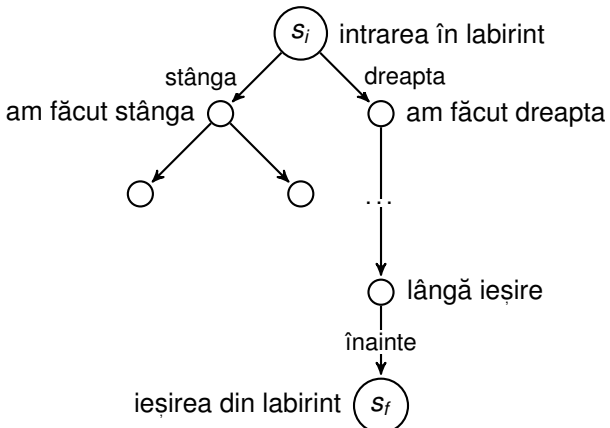
- Spațiul stărilor problemei
- Căutare
- Căutare informată
- Memorie limitată
- Exemple funcții euristice

Cum pot rezolva o problemă în care vreau să obțin, pornind de la o configurație inițială, o soluție cu anumite proprietăți?

- labirinturi
- satisfacerea restricțiilor
- jocuri
- construcția unui plan

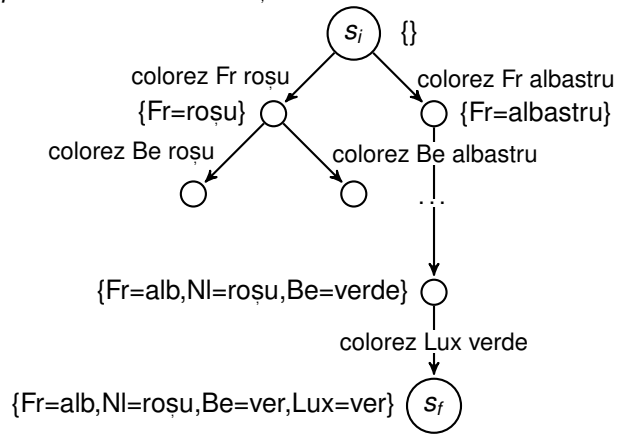
- reprezentăm problema prin graful posibilelor stări în procesul de rezolvare
- starea este schimbată prin acțiuni / mutări
- spațiu de stări $\{S\}$
- starea inițială s_i
- stări finale $\{s_f\}$
- soluție: calea $s_i \rightarrow s_f$

drumul prin labirint



- reprezentăm problema prin graful posibilelor stări în procesul de rezolvare
- starea este schimbată prin acțiuni / mutări
- spațiu de stări $\{S\}$
- starea inițială s_i
- stări finale $\{s_f\}$
- soluție: calea $s_i \rightarrow s_f$

problema colorării hărților



- Spațiul stărilor este un graf
- Pot reprezenta graful explicit sau implicit
- Rezolvarea problemei este o căutare în acest graf

Mediul în care acționăm poate fi

- Observabil sau parțial observabil
- Discret / continuu
- Finit / infinit
- Determinist / nedeterminist
- Static / dinamic

Mediul în care acționăm poate fi

- Observabil sau parțial observabil ← pot percepe orice parte din mediu la orice moment?
- Discret / continuu
- Finit / infinit
- Determinist / nedeterminist
- Static / dinamic

Mediul în care acționăm poate fi

- Observabil sau parțial observabil ← pot percepe orice parte din mediu la orice moment?
- Discret / continuu ← am o diferență clară între stări?
- Finit / infinit
- Determinist / nedeterminist
- Static / dinamic

Mediul în care acționăm poate fi

- Observabil sau parțial observabil ← pot percepe orice parte din mediu la orice moment?
- Discret / continuu ← am o diferență clară între stări?
- Finit / infinit ← este graful de stări finit?
- Determinist / nedeterminist
- Static / dinamic

Mediul în care acționăm poate fi

- Observabil sau parțial observabil ← pot percepe orice parte din mediu la orice moment?
- Discret / continuu ← am o diferență clară între stări?
- Finit / infinit ← este graful de stări finit?
- Determinist / nedeterminist ← o acțiune duce mereu în aceeași stare?
- Static / dinamic

Mediul în care acționăm poate fi

- Observabil sau parțial observabil ← pot percepe orice parte din mediu la orice moment?
- Discret / continuu ← am o diferență clară între stări?
- Finit / infinit ← este graful de stări finit?
- Determinist / nedeterminist ← o acțiune duce mereu în aceeași stare?
- Static / dinamic ← doar acțiunile agenților schimbă starea mediului

Pentru o problemă de căutare în spațiul stărilor, soluția este starea finală, sau drumul de la starea inițială la o stare finală.

- pot avea o stare inițială specificată sau nu
- pot cunoaște starea finală sau doar proprietăți ale ei
- mă poate interesa drumul către starea finală sau nu (mă interesează doar starea finală în sine)

Pot caracteriza o strategie de căutare după:

- Completitudine
- Optimalitate
- Complexitate: timp și spațiu
- Capacitate de revenire
- Informare

Pot caracteriza o strategie de căutare după:

- Completitudine ← dacă există cel puțin o soluție, atunci e garantat ca algoritmul să găsească o soluție în timp finit (nu neapărat optimă)
- Optimalitate
- Complexitate: timp și spațiu
- Capacitate de revenire
- Informare

Pot caracteriza o strategie de căutare după:

- Completitudine ← dacă există cel puțin o soluție, atunci e garantat ca algoritmul să găsească o soluție în timp finit (nu neapărat optimă)
- Optimalitate ← dacă există cel puțin o soluție, algoritmul e garantat să o găsească pe cea optimă într-un timp finit
- Complexitate: timp și spațiu
- Capacitate de revenire
- Informare

Pot caracteriza o strategie de căutare după:

- Completitudine ← dacă există cel puțin o soluție, atunci e garantat ca algoritmul să găsească o soluție în timp finit (nu neapărat optimă)
- Optimalitate ← dacă există cel puțin o soluție, algoritmul e garantat să o găsească pe cea optimă într-un timp finit
- Complexitate: timp și spațiu ← dependența spațiului de stocare și a timpului de rulare în funcție de dimensiunea intrărilor
- Capacitate de revenire
- Informare

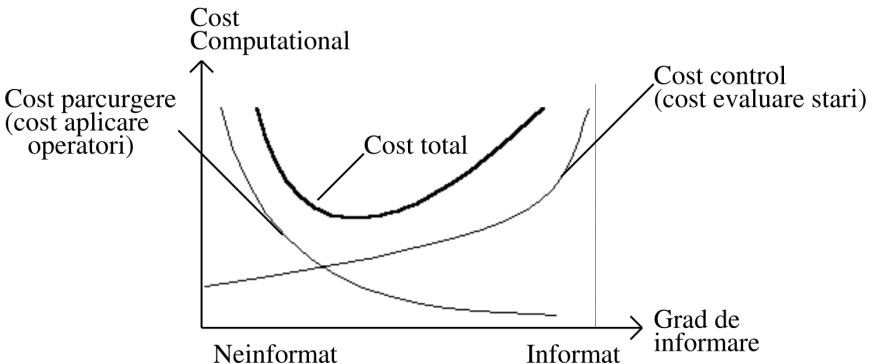
Pot caracteriza o strategie de căutare după:

- Completitudine ← dacă există cel puțin o soluție, atunci e garantat ca algoritmul să găsească o soluție în timp finit (nu neapărat optimă)
- Optimalitate ← dacă există cel puțin o soluție, algoritmul e garantat să o găsească pe cea optimă într-un timp finit
- Complexitate: timp și spațiu ← dependența spațiului de stocare și a timpului de rulare în funcție de dimensiunea intrărilor
- Capacitate de revenire ← algoritmul admite posibilitatea revizitării aceleiași stări de mai multe ori în căutare
- Informare

Pot caracteriza o strategie de căutare după:

- Completitudine ← dacă există cel puțin o soluție, atunci e garantat ca algoritmul să găsească o soluție în timp finit (nu neapărat optimă)
- Optimalitate ← dacă există cel puțin o soluție, algoritmul e garantat să o găsească pe cea optimă într-un timp finit
- Complexitate: timp și spațiu ← dependența spațiului de stocare și a timpului de rulare în funcție de dimensiunea intrărilor
- Capacitate de revenire ← algoritmul admite posibilitatea revizitării aceleiași stări de mai multe ori în căutare
- Informare ← existența unei metode de a prioritiza vecinii de explorat dintr-o stare curentă *s*

- pot avea informații despre calitatea stărilor



Pentru grafuri reprezentare explicit, putem folosi algoritmi pe grafuri:

- Dijkstra
- Bellman-Ford

Pentru grafuri reprezentare implicit:

- căutare pe lățime / în adâncime (BFS/DFS)
- Iterative Deepening
- Backtracking
- Bidirectional search

- folosim o structură pentru nodurile / stările pe care am vrea să le vizităm – Frontiera / OPEN
- folosim o structură pentru nodurile / stările pe care le-am vizitat deja (le-am explorat) și cărora le-am calculat vecinii – Domeniu / CLOSED
- când alegem un nod de explorat, îi calculăm vecinii / succesorii, în care putem ajunge executând acțiuni din starea respectivă
- adâncimea unei stări s – $Ad(s)$ – este lungimea drumului cel mai scurt din s_i până în starea s
- vectorul de părinți $\mathbf{p} \in S \times (S \cup \{\perp\})$ – părintele unei stări s este $\mathbf{p}[s]$, cu $\mathbf{p}[s_0] = \perp$

- reprezentăm soluția unei probleme de căutare în care dorim să aflăm calea către starea finală ca $\langle s_0, \ldots, s_f \rangle$
- folosim funcția $cale(s, \mathbf{p})$ pentru a obține calea de la s_0 la s

$cale(s, \mathbf{p})$

1. $sol = \langle s \rangle$
2. **cât timp** $\mathbf{p}[s] \neq \perp$
3. $\quad sol = sol \cup \mathbf{p}[s]$
4. $\quad s = \mathbf{p}[s]$
5. **întoarce** sol

Intrare: s_i, *AdMax* (pentru DFS) Ieșire: soluția, sau INSUCCES

1. OPEN $= \{s_i\}$, CLOSED $= \{\}$
2. **dacă** OPEN $= \{\}$ **atunci întoarce** INSUCCES
3. $s =$ OPEN.*pop*() extrage primul nod
4. **dacă** $s \in$ CLOSED **atunci mergi la** 2.
5. CLOSED $=$ CLOSED $\cup \{s\}$
6. [pentru DFS] **dacă** $Ad(s) =$ *AdMax* **atunci mergi la** 2.
7. $Succ =$ *calculeaza_succesori*(s)
8. **pentru fiecare** $s_j \in Succ$
9. $\mathbf{p}[s_j] = s$
10. **dacă** s_j este stare finală atunci
11. **întoarce** soluția *cale*(s_j, $\mathbf{p}$)
12. [pentru BFS] inserează s_j în OPEN la sfârșit
13. [pentru DFS] inserează s_j în OPEN la început

Algoritm Iterative deepening DFS:
Intrare: s_i Ieșire: soluția, sau INSUCCES

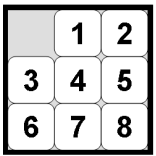
1. **pentru** $AdMax = \overline{0, \infty}$
2. $\quad rezultat = DFS(AdMax)$
3. $\quad$ **dacă** $rezultat \neq$ INSUCCES **atunci**
4. $\quad\quad$ **întoarce** $rezultat$

- B – factorul de ramificare al spațiului de căutare

e.g. pentru 8-puzzle:

- 9 poziții
- 4 colțuri – 2 mutări
- 4 centre de latură – 3 mutări
- 1 mijloc – 4 mutări

$$B = \frac{1}{9} \cdot (4 \cdot 2 + 4 \cdot 3 + 1 \cdot 4) = 2.67$$



- d – adâncimea soluției optime
- m – lungimea maximă a unei căi din spațiul de căutare

	Timp	Spațiu	Optim?	Complet?
BFS	B^d	B^d	Da	Da
DFS	B^d	$B \cdot d$	Nu	Nu
DFS/AdMax	B^m	$B \cdot m$	Nu	dacă $m \geq d$
Iterative deepening	B^d	B^d	Da	Da
Bidirectional	$B^{d/2}$	$B^{d/2}$	Da	Da

- Atenție! „Optimalitate” se referă la lungimea căii până la soluție
- Un algoritm este complet dacă găsește întotdeauna soluția, dacă aceasta există.

- informare = avem o informație asupra a cât de „bună” este o stare – cât de aproape este de soluție
- informația ne este dată de funcția $w(s, cale)$ pentru o stare s și calea din s_0 până în s
- căutare de tip *best-first*: alegem din frontieră întotdeauna starea s cu $w(s)$ *minim*

Intrare: s_i, funcția $w(s, cale)$ Ieșire: soluția, sau INSUCCES

1. OPEN $= \{s_i\}$, CLOSED $= \{\}$, $\mathbf{p}[s_i] = \perp$
2. **dacă** OPEN $= \{\}$ **atunci întoarce** INSUCCES
3. extrage $s \in$ OPEN, cu $w(s, cale(s, \mathbf{p}))$ *minim*
4. **dacă** s este stare finală **atunci întoarce** soluția $cale(s_j, \mathbf{p})$
5. CLOSED $=$ CLOSED $\cup \{s\}$
6. $Succ = calculeaza_succesori(s)$
7. **pentru fiecare** $s_j \in Succ$
8. **dacă** $s_j \in$ CLOSED $\cup$ OPEN **și**
9. $w(s_j, \{s_j\} \cup cale(s, \mathbf{p})) < w(s_j, cale(s_j, \mathbf{p}))$ **atunci** *am găsit o cale mai bună, prin s*
10. $\mathbf{p}[s_j] = s$, OPEN $=$ OPEN $\cup \{s_j\}$
11. **dacă** $s_j \notin$ CLOSED $\cup$ OPEN **atunci** *starea este nouă sau mai bună decât ce găsisem*
12. $\mathbf{p}[s_j] = s$; OPEN $=$ OPEN $\cup \{s_j\}$
13. **mergi la** 2.

- Best-first cu $w(s) = Ad(s)$ este BFS
- Best-first cu $w(s) = -Ad(s)$ este DFS
- Best-first cu $w(s_j) = \sum_{k=1}^{j} cost_arc(s_k, s_{k+1})$, cu $s_0 = s_i$ este Dijkstra
- Best-first cu $w(s) = f(s)$ este A*

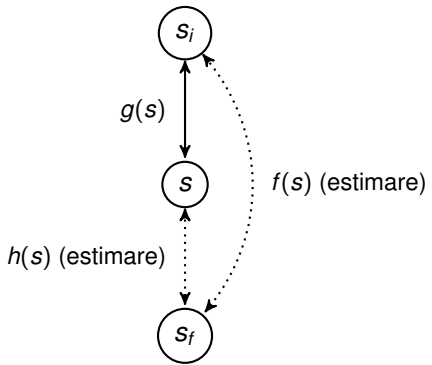
În algoritmul A* [Hart et al., 1968], starea de explorat se alege după funcția

$$f(s) = g(s) + h(s)$$

În algoritmul A* [Hart et al., 1968], starea de explorat se alege după funcția

$$f(s) = g(s) + h(s)$$

- $f(s)$ este costul total estimat al celui mai bun drum din s_0 la o stare finală, trecând prin s
- $g(s)$ este costul celui mai bun drum de la s_0 la s
- $h(s)$ este costul *estimat* al celui mai bun drum de la s la o stare finală – estimează $h^*(s)$
- $h^*(s)$ este costul real al celui mai bun drum de la s la o stare finală
- $f^*(s)$ este costul real al celui mai bun drum de la s_0 la o stare finală care trece prin s



$$g(s) = \sum_{k=1}^{j} cost_arc(s_k, s_{k+1}), \text{cu } s_0 = s_i$$

Dacă algoritmul este consistent (vezi mai jos), atunci $g(s) = g^*(s)$ este costul real al celui mai bun drum de la s_0 la s

Dacă algoritmul este admisibil (vezi mai jos), atunci $g(s) = g^*(s)$ pentru stări de pe drumul optim către starea finală.

Cum putem ajunge la un nod pe mai multe căi, vom folosi în pseudo-cod, ca și mai sus:

$$g(cale) = \sum_{\langle s_k, s_{k+1} \rangle \in cale} cost_arc(s_k, s_{k+1}), \text{ cu } cale = \langle s_0, s_1, \ldots, s_m \rangle, s_0 = s_i, s_m = s$$

$h(s)$ este o funcție euristică care *estimează* $h^*(s)$

$h(s)$ este *admisibilă* dacă...

$h(s)$ este o funcție euristică care *estimează* $h^*(s)$

$h(s)$ este *admisibilă* dacă

- $h(s) \leq h^*(s)$ (h este o euristică *optimistă*)
- $h(s_f) = 0$

$h(s)$ este o funcție euristică care *estimează* $h^*(s)$

$h(s)$ este *admisibilă* dacă

- $h(s) \leq h^*(s)$ (h este o euristică *optimistă*)
- $h(s_f) = 0$

Dacă

- funcția $h(s)$ este admisibilă, și
- $cost(s', s'') > 0, \forall s', s'' \in S$

atunci algoritmul A* este *admisibil* – este garantat că primul drum găsit de la s_i la s_f este și cel optim (din punct de vedere al costului)

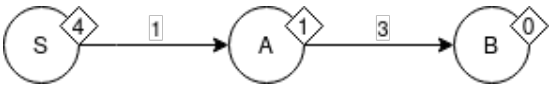
Intrare: s_i, funcția $h(s)$ Ieșire: soluția, sau INSUCCES

1. OPEN $= \{s_i\}$, CLOSED $= \{\}$, $g(s_i) = 0$, $f(s_i) = h(s_i)$, $\mathbf{p}[s_i] = \perp$
2. **dacă** OPEN $= \{\}$ **atunci întoarce** INSUCCES
3. extrage $s \in$ OPEN, cu $f(s, cale(s, \mathbf{p}))$ *minim*
4. **dacă** s_j este stare finală **atunci întoarce** soluția $cale(s_j, \mathbf{p})$
5. CLOSED $=$ CLOSED $\cup \{s\}$
6. $Succ = calculeaza_succesori(s)$
7. **pentru fiecare** $s_j \in Succ$
8. **dacă** $s_j \in$ CLOSED **și** $g(\{s_j\} \cup cale(s, \mathbf{p})) < g(cale(s_j, \mathbf{p}))$ **atunci** *am găsit o cale mai bună*
9. $\mathbf{p}[s_j] = s$, OPEN $=$ OPEN $\cup \{s_j\}$
10. **dacă** $s_j \notin$ CLOSED **atunci** *starea este nouă*
11. $\mathbf{p}[s_j] = s$
12. CLOSED $=$ CLOSED $\cup \{s_j\}$
13. OPEN $=$ OPEN $\cup \{s_j\}$
14. **mergi la** 2.

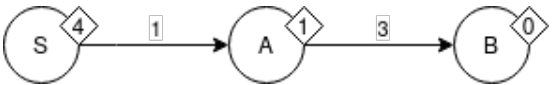
- Un algoritm **A1** este mai informat decât **A2** dacă $\forall s \in S, h_1(s) \geq h_2(s)$ (h_1 *domină* h_2)
- Atenție! $h(s) = 0, \forall s \in S$ este funcție admisibilă (și A* cu acest h devine Dijkstra)
- dacă **A1** este mai informat decât **A2**, ambii algoritmi vor descoperi soluția optimă (dacă sunt admisibili), dar **A1** va descoperi soluția optimă mai repede – explorând mai puține stări.
- o funcție euristică este *consistentă* dacă pentru $\forall s', s'' \in S$ cu s' părinte al lui s'', avem $h(s') \leq h(s'') + cost_arc(s', s'')$
  - dacă h este consistentă, atunci orice stare va fi descoperită de prima oară cu drumul cel mai bun – odată introdusă în CLOSED, nu va mai fi reintrodusă niciodată în OPEN

Putem avea euristici care să fie *admisibile*, dar să **nu** fie *consistente*?

Putem avea euristici care să fie *admisibile*, dar să **nu** fie *consistente*?

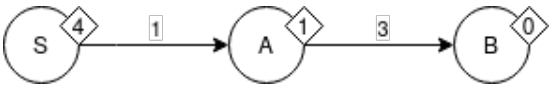


Putem avea euristici care să fie *admisibile*, dar să **nu** fie *consistente*?



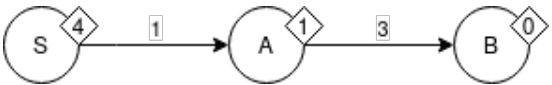
- Condiția de admisibilitate: $h(s) \leq h^*(s)$, $h(s_f) = 0$

Putem avea euristici care să fie *admisibile*, dar să **nu** fie *consistente*?



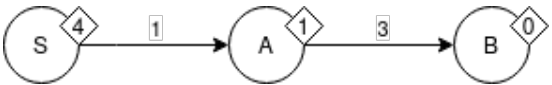
- Condiția de admisibilitate: $h(s) \leq h^*(s)$, $h(s_f) = 0$
  - $h(S) = 4 \leq 1 + 3$, $h(A) = 1 \leq 3$, $h(B) = 0 \Rightarrow$ OK

Putem avea euristici care să fie *admisibile*, dar să **nu** fie *consistente*?

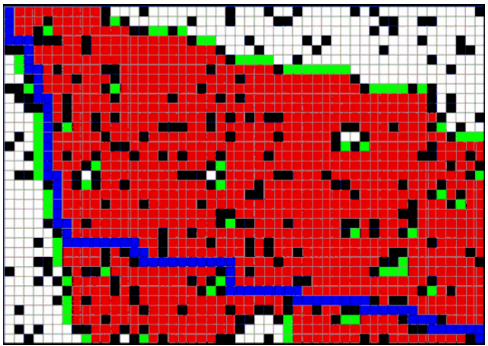


- Condiția de admisibilitate: $h(s) \leq h^*(s)$, $h(s_f) = 0$
  - $h(S) = 4 \leq 1 + 3$, $h(A) = 1 \leq 3$, $h(B) = 0 \Rightarrow$ OK
- Condiția de consistență: $\forall s', s'' \in S$ cu s' părinte al lui s'', avem $h(s') - h(s'') \leq cost_arc(s', s'')$

Putem avea euristici care să fie *admisibile*, dar să **nu** fie *consistente*?



- Condiția de admisibilitate: $h(s) \leq h^*(s)$, $h(s_f) = 0$
  - $h(S) = 4 \leq 1 + 3$, $h(A) = 1 \leq 3$, $h(B) = 0 \Rightarrow$ OK
- Condiția de consistență: $\forall s', s'' \in S$ cu s' părinte al lui s'', avem $h(s') - h(s'') \leq cost_arc(s', s'')$
  - $h(S) - h(A) = 3 > cost(S, A) = 1 \Rightarrow$ NOT OK



Wikipedia

- h este ε-*admisibilă* dacă $\forall s \in S, h(s) \leq h^*(s) + \varepsilon$

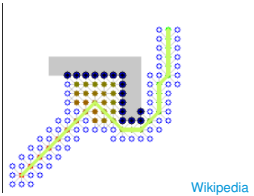
- dacă h este ε-*admisibilă*, algoritmul care utilizează această euristică găsește este ε-*admisibil* și o soluție de cost c, cu $c \leq c^* + \varepsilon$, unde c^* este costul soluției optime (soluția este ε-optimală)

- h este ε-*admisibilă* dacă $\forall s \in S, h(s) \leq h^*(s) + \varepsilon$

- dacă h este ε-*admisibilă*, algoritmul care utilizează această euristică găsește este ε-*admisibil* și o soluție de cost c, cu $c \leq c^* + \varepsilon$, unde c^* este costul soluției optime (soluția este ε-optimală)

- un algoritm A* este ponderat [Pohl, 1970] dacă favorizează euristica:

$$f(s) = g(s) + W \cdot h(s), W > 1$$



Wikipedia

- un algoritm A* ponderat va favoriza stările (estimate ca fiind) mai apropiate de scop.

- A* explorează un număr mare de stări, mai ales când euristica nu este aproape de costul real
- Există strategii de căutare care explorează un număr mai mic de stări
  - Beam search [Lowerre and Reddy, 1976]
  - Iterative deepening A* (IDA*) [Korf, 1985]
  - Memory-bound A* (MA*), Simplified MA* (SMA*) [Russell, 1992]

Memorie limitată : Beam search

- Beam search combină BFS cu o abordare *greedy* și cu utilizarea unei euristici:
- păstrează în frontieră (OPEN) doar un număr de B stări, selectate conform cu o euristică $h(s)$
- alegându-se o stare din frontieră, se adaugă toți succesorii săi la frontieră, dar apoi din frontieră se selectează doar cele mai bun B stări.
- B este *lățimea bârnei* (*beam width*)

Beam-Search(*start*, *B*, *h*, *limita*)

1. *beam* $= \{start\}$
2. *vizitat* $= \{start\}$
3. **cât timp** *beam* $\neq \emptyset$ și $|vizitat| <$ *limita*
4. $\quad$ *Succ* $= \emptyset$
5. $\quad$ **pentru fiecare** $s \in$ *beam*
6. $\quad\quad$ *Succ* $=$ *Succ* $\cup$ *calculeaza_succesori*(s) *vizitat*
7. $\quad$ **dacă** $\exists s \in$ *Succ*, s stare finală
8. $\quad\quad$ **atunci întoarce** *SUCCES*
9. $\quad$ *selectat* $=$ cele mai bune B stări din *Succ*, sortate după $h(s)$
10. $\quad$ *vizitat* $=$ *vizitat* $\cup$ *selectat*
11. $\quad$ *beam* $=$ *selectat*

Memorie limitată : Beam search

- algoritmul nu este optimal sau complet
- poate deveni complet dacă este combinat cu DFS în
  - beam stack search [Zhou and Hansen, 2005]
  - limited discrepancy search [Furcy and Koenig, 2005]
- utilizare în traducerea automată

- combină ID-DFS cu A*, limitând adâncimea explorării la o limită care folosește funcția f din A*
- explorarea se face doar până la o adâncime fixată, inițial $f(s_0)$
- când această limită este atinsă, se extinde până la $f(s^*)$, cu s^* cea mai bună stare găsită până acum

Algoritm IDA*

1. $limita = h(s_0), s = s_0, \mathbf{p}[s_0] = \perp$
2. **repetă**
3. $\quad r = DF(s, 0, limita)$
4. $\quad$ **dacă** $r = \langle s_0, \ldots, s_f \rangle$ **atunci întoarce** r *succes – a fost întoarsă o soluție*
5. $\quad$ **dacă** $r =$ INSUCCES **atunci întoarce** r
6. $\quad limita = r$ *altfel, a fost întoarsă o nouă limită*

1. $DF(s, g, limita)$
2. $\quad$ **dacă** $g + h(s) > limita$ **atunci întoarce** $g + h(s)$ *este prevăzut că soluția va depăși limita*
3. $\quad$ **dacă** s este stare finală **atunci întoarce** $cale(s, \mathbf{p})$
4. $\quad min =$ INSUCCES
5. $\quad$ **pentru fiecare** $s_j \in succesori(s)$
6. $\quad\quad$ **dacă** $s_j \notin cale(s, \mathbf{p})$
7. $\quad\quad\quad \mathbf{p}[s_j] = s$
8. $\quad\quad\quad r = DF(s_j, g + cost_arc(s, s_j), limita)$ *caută în adâncime*
9. $\quad\quad\quad$ **dacă** $r = \langle s_0, \ldots, s_f \rangle$ **atunci întoarce** r *întoarce cale*
10. $\quad\quad\quad$ **dacă** $r \neq$ INSUCCES **și** $min =$ INSUCCES **sau** $r < min$ **atunci** $min = r$
11. $\quad$ **întoarce** min

- explorează mai în direcția soluției decât DFS, pentru că zona explorată este ținută la o anumită distanță estimată față de soluție
- consumă mai puțină memorie decât A*, pentru că nu menține structurile OPEN și CLOSED

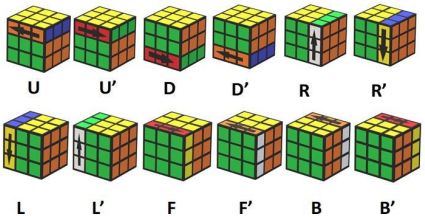
Depinde de problemă :-)

- În probleme de drum de cost optim într-un grid:
  - Distanța Manhattan
  - Distanța Euclidiană
- În problema comis-voiajorului
  - $h_1(s) = cost_arc(s_{init}, s)$
  - $h_2(s)$ = costul arborelui de acoperire de cost minim al orașelor neparcurse pana în starea s
- În jocul 8-puzzle
  - h_1 - numărul pătratelor care nu sunt pe poziția lor finală
  - h_2 / h_3 - suma distanțelor Euclidiene/Manhattan până la poziția finală pentru fiecare pătrat care nu este la locul său
  - h_4 - numărul pătratelor care nu sunt pe rândul corespunzător (poziției lor finale) + numărul pătratelor care nu sunt pe coloana corespunzătoare
  - h_5 - n-Swap: *presupunem* că "spațiul" este un pătrat de joc și că putem interschimba oricare două pătrate. Folosim costul optim al problemei "relaxate" ca euristică în problema originală.

Pattern databases

- putem reține o bază de date cu funcția euristică pentru anumite stări, care sunt soluțiile unor subprobleme ale problemei inițiale

Exemplu: pentru cubul lui Rubik, putem porni de la soluție și construi o bază de date de șabloane până la n mutări față de soluție. În momentul în care întâlnim o stare din baza de date, știm cu siguranță care este distanța minimă până la soluție [Korf, 1997]



source

Mulțumesc!



https://forms.gle/DJUdkejstkvyNRF5A

Feedbackul este binevenit!


Furcy, D. and Koenig, S. (2005).
Limited discrepancy beam search.
In *IJCAI*, pages 125–131.


Hart, P. E., Nilsson, N. J., and Raphael, B. (1968).
A formal basis for the heuristic determination of minimum cost paths.
IEEE transactions on Systems Science and Cybernetics, 4(2):100–107.


Korf, R. E. (1985).
Depth-first iterative-deepening: An optimal admissible tree search.
Artificial intelligence, 27(1):97–109.


Korf, R. E. (1997).
Finding optimal solutions to rubik's cube using pattern databases.
In *AAAI/IAAI*, pages 700–705.


Lowerre, B. and Reddy, R. (1976).
The harpy speech recognition system: performance with large vocabularies.
The Journal of the Acoustical Society of America, 60(S1):S10–S11.


Pohl, I. (1970).
First results on the effect of error in heuristic search.
Machine Intelligence, 5:219–236.


Russell, S. (1992).
Efficient memory-bounded search methods.
In Neumann, B., editor, *10th European Conference on Artificial Intelligence, ECAI 92, Vienna, Austria, August 3-7, 1992. Proceedings*, pages 1–5. John Wiley and Sons.


Zhou, R. and Hansen, E. A. (2005).
Beam-stack search: Integrating backtracking with beam search.
In *ICAPS*, pages 90–98.